

Weight Custody Manifest: Portable, Attestation-Gated Custody for Proprietary Model Weights

Imran Siddique
OPAQUE Systems

<https://github.com/agentrust-io/weight-custody-manifest>

September 27, 2026
Technical report, version 1

Abstract

Deploying proprietary model weights into infrastructure controlled by a customer reverses the usual confidential-computing trust problem: the model builder, rather than the infrastructure owner, supplies the secret. Existing confidential-computing stacks can attest workloads and release secrets, while model-signing systems authenticate model artifacts. Neither capability alone specifies continuing custody of a particular weight artifact after release. This technical report describes the Weight Custody Manifest (WCM), a vendor-neutral protocol and conformance model that binds an exact weight digest, an approved workload measurement, attestation policy, a fresh transport key, renewable authorization, terminal refusal and wipe evidence, and derivative lineage. WCM distinguishes cryptographic custody against software adversaries from accountability-grade controls when the infrastructure operator physically owns the machine. The evaluated Python reference implementation, version 0.28.0, supplies 91 portable conformance vectors across four levels. Its test suite produced 618 passing tests and three skips on September 3, 2026, and the same counts when rerun at the same commit for this edition. Two later findings bound those results: a published advisory showed that the evaluated key broker did not require GPU confidential-compute mode before release, and the SEV-SNP platform used for the hardware run does not enable the ciphertext hiding that the cryptographic-custody claim against a hypervisor-privileged operator requires. The contribution is a portable artifact-to-runtime custody contract with explicit lifecycle evidence, derivative accountability, and conformance, not a new attestation primitive or a certification of any deployment.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. It revises a manuscript dated September 3, 2026. The software results were rerun on September 27, 2026 at the evaluated commit (2acedfa, WCM 0.28.0) with current dependency resolutions; the hardware runs were not repeated. Findings published after the evaluation are reported in Section 6 beside the results they qualify. Current behavior, normative requirements, and limitations are maintained in the project repository.

Patent Pending.

1 Introduction

Frontier-model deployment increasingly separates the model builder from the infrastructure operator. A builder may need to run a closed model inside a customer’s cloud account, sovereign environment, or on-premises cluster without giving the operator a reusable plaintext copy of the weights. Encryption at rest is insufficient because an inference runtime eventually needs a decryption capability. Remote attestation and key brokers address the initial release decision: a verifier appraises evidence from a protected workload, and a relying party releases a secret only when policy is satisfied [Birkholz et al., 2023, Confidential Containers, 2026, NVIDIA, 2026].

The initial release is only one boundary in a longer custody lifecycle. A deployer must identify the exact model bytes, bind them to the intended workload and policy, prevent replay of old evidence, communicate the released key only to the attested instance, renew or revoke authority, stop serving when authority lapses, preserve tamper-evident evidence, and account for derived artifacts. Existing systems implement important subsets, but their deployment-specific policy and secret-delivery interfaces do not constitute a portable model-custody record.

WCM contributes a composition rather than a new attestation primitive. Its signed manifest is the stable object that a model builder and independent verifier can share across providers. It names exact artifacts and allowed measurements, constrains key release, and connects the release decision to runtime custody and lineage records. Four design rules follow:

1. an artifact signature is necessary but does not authorize execution;
2. attestation evidence is necessary but does not identify which weight artifact may be released;
3. a successful release is temporary, so continued use requires renewable authority and observable terminal behavior;
4. physical ownership of current confidential-computing hardware is outside the strongest cryptographic guarantee.

2 Threat model and guarantee boundary

The protected asset is the plaintext weight artifact and its release key. The model builder controls the artifact digest, signing key, acceptable workload measurements, release policy, and revocation decisions. The customer or platform operator controls scheduling, storage, networking, host software, and availability. The protected runtime and verifier form part of the trusted computing base; the host operating system and remote software administrator need not.

WCM defines two non-interchangeable claims. Cryptographic custody targets remote and software adversaries, including a compromised host stack, provided the selected TEE and its verified configuration actually defend that adversary. Accountability-grade custody is the maximum claim against an operator with physical possession of the machine. TEE.fail demonstrates inexpensive DDR5 interposition attacks against contemporary TDX and SEV-SNP systems and shows consequences for GPU confidential computing [Chuang et al., 2026]. BadRAM similarly restores an attestation break against updated SEV-SNP systems [De Meulemeester et al., 2025]. WCM therefore records the physical-control tier and residual risk rather than transforming a signed quote into a claim of physical impossibility.

Denial of service, malicious model code, side channels outside the selected profile, compromised builder keys, and truthful-but-misleading application outputs remain out of scope. WCM does not certify an entire organization’s RAND security level; RAND levels describe whole-system postures, not one control [Nevo et al., 2024].

3 Protocol

3.1 Manifest and artifact identity

The model builder computes a deterministic digest over the weight artifact and signs a manifest containing the artifact identity, allowed workload measurements, evidence requirements, release-key identifier, validity interval, renewal policy, and lineage policy. Model signing systems such as OpenSSF Model Signing establish artifact authenticity and integrity [OpenSSF, 2026]; WCM consumes that result and adds a runtime authorization decision. A valid artifact signature does not imply that the current workload may decrypt the artifact.

3.2 Fresh attestation and channel binding

The key broker issues a fresh single-use challenge. The protected workload generates an ephemeral transport key and obtains CPU, and when required GPU, evidence that binds the challenge, workload state, and transport key. The verifier validates certificate chains, freshness, revocation, measurements, and policy. The separation follows the RATS architecture: evidence is appraised by a verifier, and the relying party applies its own policy to the resulting claims [Birkholz et al., 2023].

The key broker releases only when the verified evidence and manifest policy agree. Release material is sealed to the attested transport key; a plaintext model key is never returned merely because a caller presents a valid quote. Confidential Containers Trustee and NVIDIA reference architectures already demonstrate the underlying attestation-and-secret-delivery pattern [Confidential Containers, 2026, NVIDIA, 2026]. WCM standardizes the model-specific manifest, joins, lifecycle, and evidence surrounding that pattern.

3.3 Renewable custody

An authorization lease expires independently of process lifetime. Before expiry the runtime must obtain a new challenge and satisfy current release policy. Failure or revocation transitions the reference state machine to a terminal condition: inference is refused and the in-memory key is overwritten on a best-effort basis. The signed runtime-record chain makes the claimed sequence tamper-evident and independently replayable. It does not prove language-runtime or silicon-level zeroization.

3.4 Derivative lineage

Fine-tunes, adapters, merges, and quantized variants may encode economically valuable information from a protected parent. WCM records parent digests, transformation identity, custody policy, and resulting artifact digest. The lineage is evidence of declared derivation and policy continuity, not a proof that an undeclared derivative does not exist.

4 Conformance and implementation

The reference project includes a JSON schema frozen at version 1, a Python SDK, a reference key-broker service, hardware evidence adapters, and language-neutral conformance vectors. Ninety-one vectors are organized into four levels: document integrity and signatures; attestation-gated release scenarios; renewable runtime-custody scenarios; and lineage and accountability records. Each reportable error code is exercised. The runner states two important limitations on every execution: its portable PKI is synthetic rather than a set of live vendor roots, and it contains no GPU-side cryptographic vectors.

The implementation checks RFC 8785 canonicalization, certificate and signature chains, challenge freshness, approved measurements, transport-key binding, revocation, threshold policy, transparency receipts, derivative lineage, and terminal runtime records. Provider adapters cover SEV-SNP, TDX, Azure vTPM, and NVIDIA evidence surfaces. Integration profiles connect WCM to Confidential Containers Trustee, NVIDIA NRAS, cloud confidential-computing services, OCI, in-toto, Kyverno, vLLM, Triton, OpenTelemetry, Agent Manifest, and TRACE. These adapters are evidence of composability; they are not all equivalent hardware validations.

5 Evaluation

We evaluate three questions.

E1: Does the portable verifier fail closed? The conformance suite includes valid vectors and mutations across schema, signature, digest, freshness, policy, evidence, release, renewal, and lineage boundaries. On September 3, 2026, the complete Python suite at WCM 0.28.0 produced 618 passing tests and three skips; a rerun at the same commit on September 27, 2026 produced the same counts, and the reference conformance self-test passed all 91 vectors (L1 32, L2 37, L3 12, L4 10). The three skipped tests exercise symbolic links

in artifact digests and were skipped because the Windows test account lacked the privilege to create them; they are not counted as passes. The original manuscript described them as hardware-dependent, which was incorrect. This result checks the behavior the vectors encode. It does not show that the vectors encoded every release requirement: Section 6 describes a GPU requirement they omitted.

E2: Is custody state observable after release? Tests exercise successful release, renewal, lapse, refusal, and terminal signed records. A recorded protected-runtime experiment executed a lease to lapse inside an Azure SEV-SNP guest, refused subsequent service, wiped the Python-held key buffer, and produced a chain that verifies offline. The bounded conclusion is that the reference state machine and evidence chain behaved as specified. It does not establish physical non-recoverability.

E3: Can the design expose rather than hide the hardware boundary? A protected-memory sweep was captured over 256 MB inside a SEV-SNP guest and bound to live vTPM evidence. The sweep detects controlled alias mappings within its observation boundary. Its signed result also states that an in-guest sweep cannot observe an external memory interposer. This negative claim is part of the artifact, so detached evidence retains its limitation.

Property	Evidence	Bounded conclusion
Portable behavior	91 vectors; 618 pass, 3 skip (0.28.0)	behavior the vectors encode
Fresh release	nonce and transport-key tests	replay/substitution rejected
Renewal/lapse	custody state-machine tests	terminal refusal is reproducible
Hardware run	signed SEV-SNP runtime records	one measured run, not certification
Physical owner	explicit residual-risk field	accountability, not impossibility

Table 1: Evaluation evidence and its permitted interpretation.

6 Findings after the evaluation

Three findings published after September 3, 2026 narrow the conclusions above. They are recorded in the project’s security advisory and limitations document.

GPU confidential-compute mode was not a release condition. Advisory GHSA-j665-99rh-w85h (high severity, published September 25, 2026) reported that in versions up to 0.28.2 the key broker populated the GPU report’s confidential-compute mode but no release gate read it, and the NVIDIA provider defaulted a missing value to enabled. A confidential-computing-capable GPU with that mode disabled could therefore receive the model key. The specification had the same gap. Version 0.28.4 adds a signed requirement and a corresponding error code. The evaluated version, 0.28.0, is affected, so the fail-closed result in E1 does not extend to GPU mode.

The measured SEV-SNP platform lacks ciphertext hiding. The cryptographic-custody claim against a hypervisor-privileged operator depends on SEV-SNP ciphertext hiding, because without it ciphertext side channels can expose keys without physical access. The Azure confidential VM used for E2 and E3 reports PLATFORM_INFO 0x25: alias checking complete, ciphertext hiding not enabled. A later GCP SEV-SNP capture reports the same state. On these platforms the claim holds only against adversaries without hypervisor privilege. The evaluated version did not read this field; later versions can require it in signed policy.

Attestation-key revocation is weak in practice. A check made on September 11, 2026 found that the captured NVIDIA device chain carries certificates valid to 9999 with empty long-lived revocation lists, that AMD VCEK certificates carry serial number zero, and that an operator has no documented path to revoke a specific device key. A passing revocation check shows only that the vendor has published no revocation.

7 Related work and novelty boundary

Confidential computing for machine learning is a mature research area; prior systems protect models or data within TEEs, and the literature documents both security benefits and platform limitations [Mo et al.,

2022, Vaswani et al., 2022]. Trustee provides reusable attestation, reference-value, KBS, and secret-delivery components [Confidential Containers, 2026]. NVIDIA’s architecture applies those concepts directly to proprietary models [NVIDIA, 2026]. OpenSSF Model Signing authenticates model artifacts [OpenSSF, 2026]; in-toto already supplies signed artifact subjects, predicates, and supply-chain provenance [in-toto Project, 2026]; and CryptoTensors applies encryption and access policy to tensor artifacts [Zhu et al., 2025]. WCM neither replaces nor precedes these systems.

The narrower contribution is a vendor-neutral custody object and conformance boundary spanning exact artifact identity, workload authorization, freshness, channel-bound release, renewable post-release state, terminal evidence, and derivative lineage, with a first-class downgrade for physical ownership. This claim is falsifiable: discovery of a prior portable specification and interoperable implementation covering that complete conjunction would require the contribution to be narrowed.

8 Limitations and conclusion

WCM remains pre-1.0. The reference implementation is not a production certification. Vendor evidence quality varies, physical attacks can invalidate assumed roots of trust, and signed records prove only what their trusted producers could observe. The implementation’s zeroization behavior is best effort at the Python layer. Independent implementations and external conformance runs are still needed.

WCM turns a deployment recipe into a portable, testable custody contract without overstating what present hardware can guarantee. Its principal result is the lifecycle join: which exact weights were authorized for which measured workload, through which fresh and channel-bound release, under which renewable policy, with what terminal and derivative evidence, and under which explicitly bounded adversary model.

References

- Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote ATtestation procedureS (RATS) architecture. Technical Report RFC 9334, IETF, 2023. URL <https://www.rfc-editor.org/rfc/rfc9334.html>.
- Jalen Chuang, Alex Seto, Nicolas Berrios, Stephan van Schaik, Christina Garman, and Daniel Genkin. TEE.fail: Breaking trusted execution environments via DDR5 memory bus interposition. In IEEE Symposium on Security and Privacy, 2026. URL <https://tee.fail/>.
- Confidential Containers. Trustee: Attestation and secret delivery components, 2026. URL <https://github.com/confidential-containers/trustee>.
- Jesse De Meulemeester, Luca Wilke, David Oswald, Thomas Eisenbarth, Ingrid Verbauwhede, and Jo Van Bulck. BadRAM: Practical memory aliasing attacks on trusted execution environments. In IEEE Symposium on Security and Privacy, 2025. URL <https://badram.eu/>.
- in-toto Project. in-toto attestation framework, 2026. URL <https://github.com/in-toto/attestation>.
- Fan Mo, Zahra Tarkhani, and Hamed Haddadi. Machine learning with confidential computing: A systematization of knowledge. arXiv preprint arXiv:2208.10134, 2022. URL <https://arxiv.org/abs/2208.10134>.
- Sella Nevo, Dan Lahav, Ajay Karpur, et al. Securing AI model weights: Preventing theft and misuse of frontier models. Technical Report RRA2849-1, RAND Corporation, 2024. URL https://www.rand.org/pubs/research_reports/RRA2849-1.html.
- NVIDIA. Deploying proprietary models securely with NVIDIA confidential computing, 2026. URL <https://docs.nvidia.com/enterprise-reference-architectures/deploying-proprietary-models-confidential-compute-self-hosted-vms/latest/architecture-summary.html>.
- OpenSSF. OpenSSF model signing, 2026. URL <https://openssf.org/projects/model-signing/>.

Kapil Vaswani, Stavros Volos, Cédric Fournet, Antonio Nino Diaz, Ken Gordon, Balaji Vembu, et al. Confidential machine learning within Graphcore IPUs. arXiv:2205.09005, 2022. URL <https://arxiv.org/abs/2205.09005>.

Huifeng Zhu, Shijie Li, Qinfeng Li, and Yier Jin. CryptoTensors: A light-weight large language model file format for highly-secure model distribution. arXiv:2512.04580, 2025. URL <https://arxiv.org/abs/2512.04580>.